Practice Questions

Assembly Language and Computer Organization

Contents

0.1. Bit Manipulation and Flag Operations

Q1. Bitwise Shift and Rotate Operations

Given the 8-bit value `AL = 10110100b`, determine the new value of AL and the Carry Flag (CF) after each of the following single operations:

a) SHR AL, 1

b) SHL AL, 1

c) SAR AL, 1

d) ROR AL, 1

e) ROL AL, 1

Q2. Rotate Through Carry

If `AL = 11001010b` and `CF = 1`, show the result in AL and the updated Carry Flag after:

a) RCR AL, 1

b) RCL AL, 1

Explain how the carry flag participates in both operations.

Q3. Bit Reversal

Write a short Assembly routine that reverses all bits of AL (MSB $\leftrightarrow$ LSB).
Example: AL = 10110010b $\rightarrow$ AL = 01001101b

Q4. Extracting a Specific Bit

Using only shift and rotate instructions, extract bit 5 from the 8-bit register AL and display its value.

Q5. Bit Reversal in Nibbles

Write an Assembly program that reverses the bits inside each nibble of a given 16-bit number.
Example: 1011 0101 1100 0110 $\rightarrow$ 1101 1010 0011 0110

Q6. Swap Every Pair of Bits in AX

Develop a program that swaps every pair of bits in the AX register.
Example: AX = 1011 0100 0001 1110b → 0111 1000 0101 1101b
Use only `AND, OR, XOR, SHL, SHR, ROL, ROR, RCL, RCR, SAL, SAR` instructions.

Q7. Nibble Swap in AX

Develop a program to swap the upper and lower nibble of each byte in AX.
Example: AX = ABh → BAh; AX = 1234h → 2143h.

Q8. Bit Counting and Complementation

AX and BX contain 16-bit values. Generate a program to count the number of 1 bits in BX, then complement the same number of least significant bits in AX.

Q9. Complement Specific Bit

AX contains a value between 0–15. Complement the corresponding bit in BX (e.g., if AX=6, toggle the 6th bit of BX). Show before and after values of BX in memory.

0.2. Conditional and Logical Operations

Q10. Even/Odd Bit Modification

Take an 8-bit number stored in AL and perform the following using only `AND, OR, XOR`:

1. If the number is even, set its MSB, 6th, and 5th bits, and take the 1's complement of all remaining bits.
2. If the number is odd, clear its MSB, 7th, and 3rd bits, and take the 1's complement of all remaining bits.
3. Finally, take the 2's complement of the result and store it in memory.

Q11. Array Bit Checking

Take an array of 8 numbers. Check and count how many of these have their 5th bit set. Toggle all other bits of that number and store the count in memory. (All operations must be performed in a subroutine.)

0.3. Arithmetic and Number Logic

Q12. Fibonacci Series

Write an Assembly program to generate the Fibonacci series up to n terms.
Example: n=6 → 0 1 1 2 3 5

Q13. Armstrong Number

Write an Assembly program to check whether a number is an Armstrong number.
Example: 153 → Armstrong, 123 → Not Armstrong.

Q14. Prime Factors

Write an Assembly program to find the prime factors of a given number.
Example: 60 → 2 2 3 5

Q15. Coin Change (Fewest Coins)

Write an Assembly program that gives the fewest coins for change.
Example: Change 40 → 25 10 5

Q16. Binary Palindrome Checker

Write an Assembly program to check if a binary number is a palindrome.
Example: 1001 → Palindrome, 1010 → Not Palindrome.

Q17. Iterative Fibonacci (Subroutine Version)

Develop an iterative Fibonacci subroutine that:

- Takes number n from memory,
- Returns Fibonacci(n) in AX,
- Uses two registers to hold current and previous numbers.

Hint:

```
push word [num]
call fibonacci_iter
```

Q18. Factorial

Write a program to compute the factorial of a number.
Example: Input = 5 → AX = 120

0.4. Data Handling and Sorting

Q19. Bubble Sort (Descending Order)

Given:

```
data dw 25, 60, -15, -10, 50
```

Write a program fragment to sort this signed array in descending order using the Bubble Sort algorithm.

Q20. Signed vs Unsigned Sorting

Given:

```
data db 120, 100, -10, 5, -100
```

Predict the sorted output if the program uses:

- JA/JB (unsigned jumps)
- JG/JL (signed jumps)

Q21. Sorting Logic Modifications

a) Which single instruction change will make Bubble Sort descending?

b) How can you make it sort only the first 4 elements of an 8-element array?

c) Where will you add a pass counter to track total passes?

Q22. Duplicate Handling

If two consecutive elements are equal (e.g., [50, 50, 30, 10]):

- What happens if JB is used instead of JBE?
- How does this affect swaps and final order?

Q23. Early Exit Optimization

Explain the purpose of:

```
mov byte [swap], 0
```

Discuss:

- What happens if removed?
- How does it affect efficiency for an already sorted array?

0.5. System and Addressing Concepts

Q24. Memory Capacity Check

A computer has 30 address lines and a 16-bit data bus. Can it fully address 4 GB memory? Justify.

Q25. Data Definition Problem

```
mov al, [num1]
num1: dw 1234h
```

a) Identify the problem in Instruction A.

b) Rewrite correctly to:

(I) Load only lower byte (34h)

(II) Load full word (1234h)

(III) Load higher byte (12h)

Q26. SHR and Signed Numbers

Explain (with example) why using SHR on a signed integer gives an incorrect mathematical result.

Q27. CMP Flags and Jumps

Given:

```
AX = 0A23h
BX = 0A23h
cmp ax, bx
```

a) Predict the status of ZF, SF, CF.

b) Which jumps will be taken (je, jne, jg, jl, ja, jb)?

c) Write short code to display whether numbers are equal, greater, or smaller.

0.6. Miscellaneous Assembly Tasks

Q28. Count Negatives, Zeros, Positives

Write a program to accept five signed numbers in an array. Count how many are negative, zero, and positive. Store counts in `neg_count`, `zero_count`, and `pos_count`.

Q29. Binary Search

Extend the binary search program to:

- Search an element in a sorted array of 10 numbers.
- If found → store index in BX and set ZF=1.
- If not found → BX = FFFFh, ZF=0.

Q30. Linked List Node Management

Declare 4 nodes of 4 bytes each:

- First 2 bytes = data
- Next 2 bytes = offset of next node

Implement procedures for:

- **init** → chain all nodes
- **insertafter(node, data)** → insert new node after given one

Q31. Replace Instruction Sequences

Replace each sequence with a single instruction (ignore flag effects):

a)
```
push word L1
jmp L2
L1:
```

b)
```
sub sp, 2
mov [ss:sp], ax
```

c)
```
mov ax, [ss:sp]
add sp, 2
```

Q32. Iterative Bit Counting

AX contains a non-zero 16-bit number. Develop a program that:

- Counts the number of 1-bits in AX,
- Replaces AX with that count,
- Repeats until AX = 1,
- Stores in BX the number of iterations it took.

Q33. Population Count Parity in Array

Write an assembly language program that takes an array of 5 numbers and counts how many numbers in the array have an even number of 1-bits and how many have an odd number of 1-bits in their binary representation.

Example:

```
Array: 12 (1100b → 2 ones → even),
       45 (101101b → 4 ones → even),
       7 (111b → 3 ones → odd),
       255 (11111111b → 8 ones → even),
       170 (10101010b → 4 ones → even)
```

Output: Even = 4, Odd = 1

Requirements:

- Use 16-bit registers and process each number bit-by-bit.
- Store final counts in memory variables `EVEN_COUNT` and `ODD_COUNT`.
- Display results using DOS interrupt `INT 21h`.

Full Updated LaTeX Structure (Q1–Q32 remain, Q33 is new, Q34–Q60 removed): latex

Q33. Population Count Parity in Array

Write an assembly language program that takes an array of 5 numbers and counts how many numbers in the array have an even number of 1-bits and how many have an odd number of 1-bits in their binary representation.

Example:

```
Array: 12 (1100b → 2 ones → even),
       45 (101101b → 4 ones → even),
       7 (111b → 3 ones → odd),
       255 (11111111b → 8 ones → even),
       170 (10101010b → 4 ones → even)
```

Output: Even = 4, Odd = 1

Requirements:

- Use 16-bit registers and process each number bit-by-bit.
- Store final counts in memory variables `EVEN_COUNT` and `ODD_COUNT`.
- Display results using DOS interrupt `INT 21h`.